

Refactoring Detection based on UMLDiff Change-Facts Queries

Zhenchang Xing and Eleni Stroulia
Computing Science Department
University of Alberta
Edmonton AB, T6G 2H1, Canada
{xing, stroulia}@cs.ualberta.ca

Abstract

Refactoring is an important activity in the evolutionary development of object-oriented software systems. Several IDEs today support the automated application of some refactorings; at the same time, there is substantial on-going research aimed at developing support for deciding when and how software should be refactored and for estimating the effect of the refactoring on the quality requirements of the software. On the other hand, understanding the refactorings in the evolutionary history of a software system is essential in understanding its design rationale. Yet, only very limited support exists for detecting refactorings. In this paper, we present our approach for detecting refactorings by analyzing the system evolution at the design level. We evaluate our method with case studies, examining two realistic examples of object-oriented software.

1 Introduction

Capturing and maintaining design rationale has been a long-term goal of several different methods developed in support of different activities [7], from requirements tracing to design [20], documentation [25] and maintenance [3,4]. These methods aim at recording and maintaining information about why developers have made the decisions they have, so that it can be used to ease further development and improve the quality of future decisions by increasing their consistency with past decisions [14].

Today, as much of software is developed using some evolutionary lifecycle process, the software design rationale is embedded in the “evolution decisions” of the developers, i.e., the changes they have made to the system from its first version to its current state. Therefore, the objective of maintaining software-design rationale becomes that of understanding *what* is the sequence of changes that the system has undergone to reach its current state and *why*.

There has been a substantial amount of work on reverse engineering different aspects of the change history of software systems from the information stored in

software repositories [6,8,9,17]. Among all changes, refactorings are especially interesting. In the context of object-oriented software development, interesting design decisions are often accomplished by means of refactorings, which involve reorganizing the containment or inheritance hierarchies, moving features among classes, restructuring of data structures or class interfaces, changes to the interactions between classes and so on. Such decisions, to implement a particular refactoring in some part of the system, are reflected in the source code and in the way the code has changed over time.

Indeed, researchers have examined many aspects of software code evolution to infer which refactorings might have been applied to the system. These methods have been based on examining change documentation [5], or comparatively analyzing source-code metrics [6], or clone detection [12,22], or visualization [13]. Unfortunately, all these approaches suffer from some non-trivial disadvantages. More frequently than not, there is no consistently maintained change documentation. Aggregate code metrics do not provide sufficient information to precisely pinpoint the elements involved in the refactoring. Clone detection is not very effective in recognizing “non-local” refactorings that involve several entities. Finally, visualization techniques do not scale well as the size of the system increases.

In our work, we have adopted a design-level differencing methodology to recognizing refactoring decisions. The key is to recover the changes in the context in which they were made. Thus, our approach to detecting and classifying refactorings is based on the design-level changes reported by the *UMLDiff* algorithm [27].

UMLDiff is a domain-specific structural differencing algorithm that automatically detects (a) additions, removals, moves, renamings of subsystems, packages, classes, interfaces, fields and methods, (b) changes to their attributes, such as data type, visibility, and deprecation-status, and (c) changes of the containment, inheritance, and usage dependencies among these entities.

Based on the elementary structural changes produced by *UMLDiff*, we have defined queries to automatically detect more complex structural change patterns, as compositions of elementary changes, such as the composite refactorings [10] listed in Fowler's refactoring catalog, or other general structural-change patterns of interest. The instances of refactorings are reported in terms of their particular types and their participants. Finally, the changes reported by *UMLDiff* are visualized in inheritance or containment *change-trees*, i.e., a tree of structural changes, to facilitate the analysis of the underlying rationale and motivation behind the refactorings.

Our case studies, examining independently developed software systems, demonstrate that our approach is applicable in practice and can effectively focus the developers' attention on the relevant parts of the system. The recovered concrete instances of structural change patterns and their participants can point them to the changing places where the design has evolved. We believe that the very process of recognizing and reflecting upon concrete interesting instances of refactorings in the evolutionary history of the software system can help developers understand the system design and its evolution so that it can be consistently evolved.

The remainder of this paper is structured as follows. Section 2 relates this work to previous research. Section 3 presents an overview of our methodology. Sections 4 and 5 evaluate our approach through two case studies and argue for its usefulness. Concluding remarks are outlined in Section 6.

2 Related work

Refactoring has recently become an integral part of the evolutionary software development methodology, such as "Extreme Programming" [2]. The research area of refactoring was pioneered by Opdyke [19]. The books of Fowler [10] and Kerievsky [16] provide a good overview of the refactorings and how they can be used to accomplish architectural and design changes.

Although refactoring is usually a developer-intensive activity, tool support is considered crucial [1,15]. Opdyke developed the first tool for providing semi-automatic refactoring support, which was implemented in the Refactoring Browser [21]. Modern IDEs, such as Eclipse [30] and IntelliJ IDEA [35], offer semi-automatic support for most commonly used refactorings.

There has also been some work on reverse engineering refactorings by mining software repositories. Demeyer et al. [6] defined four heuristics based on the comparison of source-code metrics of two subsequent system snapshots to identify refactorings of three general categories. Rysselberghe and Demeyer [22] investigated the use of clone detection to identify "move" and "renaming" refactorings. Godfrey and Zou [12]

used origin analysis to detect the "merging" and "splitting" of source-code entities.

Dig and Johnson [5] conducted an empirical study to assess the role of refactorings in API migration. Their analysis relies on the changes documented in the release notes shipped with software systems. Gorg and Weisgerber [13] analyzed the changes of entity signatures in evolution transactions – i.e., changes to a set of files committed at the same time by the same developer – to detect refactorings, such as "rename method", "move class", "pull-up method". Their approach is sensitive to the "multiple refactorings on the same location".

In our previous work [26], we sketched out a technique for recognizing refactorings from the design-level structural changes and we investigated the cross-reference of four types of information (code metrics, identifiers, design differencing, and return-on-investment) for inferring refactoring activities [23].

In this paper, we report on our fully implemented approach for detecting refactorings. It relies on a novel structural-differencing algorithm, *UMLDiff* [27], which automatically detects the design-level structural changes in more detail than what is covered in the documentation and more intuitive than source-code metrics. Refactorings are identified according to their effects on the design entities and their inter-relationships and are reported in terms of their particular types and participants.

The case studies reported in this paper show that our approach is able to identify a broad set of refactorings and is scalable and robust. The identification of instances of a wide-range of refactorings provides us the opportunity to study in depth the refactoring practice in the real-world software development to understand how it is and how it should be supported [31]

3 The JDevAn Refactoring Detection Process

The JDevAn – "Java Design Evolution and Analysis" – tool [36] analyzes the evolving structure of design-level software artifacts and automatically produces reports of interesting trends and events during their evolution, such as refactorings that we discuss in this paper. In this section, we briefly discuss the basic JDevAn functionalities relevant to recognizing refactorings. Interested readers are referred to [27,28,29,36] for more details.

3.1 Logical-Design Facts

JDevAn's fact extractor (based on the Eclipse DOM/AST model) recovers a model of the subject system's class design (according to the semantics of the UML static-structure model [18]).

The extracted facts describe program entities of the types supported by the Java programming language,

i.e., (a) package (including subsystem), (b) (anonymous) class, (c) interface, (d) array type, (e) primitive type, (f) field and (g) block (including method, constructor and initializer). Each extracted entity is described in terms of (a) the version of the system in which it belongs, (b) its location in the source file, (c) its type and name, (d) visibility and non-access modifiers, (e) whether it is deprecated, and (f) whether it belongs in the system source code or in a library.

In addition, facts about the relations among these entities are extracted, including (a) ownership (containment and declaration), (b) generalization and abstraction (interface implementation), (c) usage dependency (field read/write, method call (including this, super, and constructor call), exceptions thrown and caught by a method, and class instantiation), (d) field data type, method return type, and parameter type, and (e) exceptions declared by a method. The outgoing usage dependencies of anonymous class's fields and methods are populated to their corresponding containing methods. Such indirect outgoing usage of the containing methods is tagged to indicate that they originate from their contained anonymous classes.

The extracted software-system models are stored in a PostgreSQL relational database, extended with Simon's transitive closure algorithm [24] for computing the transitive closure of the containment and inheritance hierarchies, field read/write and method calls, and class/interface usage relations.

3.2 Logical-Design Change Facts

UMLDiff [27] is JDevAn's UML-aware design-level structural differencing algorithm. Given the facts extracted from two subsequent system versions (before and after), *UMLDiff* establishes that two entities of the same UML type, belonging to each of the compared system versions, represent the "same" conceptual entity, i.e., they are "mapped" (either "matched", "renamed", or "moved"), if they have the same or similar names and if they have similar structural relations to other entities that have already been established to be the same. Mapped entities can still differ from each other, since some of their attributes and dependencies may have changed. If some entity in the before system version is not mapped to any entity in the later version, it is considered to have been "removed" from the system. If some entity in the after system version is not mapped to any entity in the earlier one, it is considered to have been "added" to the system in this later version.

Table 1 summarizes the types of changes *UMLDiff* reports for different types of entities. *UMLDiff* recovers design changes in the context in which they were made.

The changes *UMLDiff* reports are stored in JDevAn's database in the form of $\langle scategory, stype, prev, next \rangle$ tuples: *scategory* and *stype* represent the category and type of the change; *prev* and *next* are in-

teger arrays containing information about the entities involved in the change in the before and after versions respectively, including the ids of the involved entity, and, if necessary, their attributes.

Table 1. The summary of *UMLDiff* changes

Entity type	Categories and types of <i>UMLDiff</i> changes
Subsystem & package	Added, removed, renamed, or moved
Class & interface	- Added, removed, renamed, or moved - Generalization change of class and interface, and no-longer or new interface implementation of class - No-longer or new outgoing and incoming usage dependencies - Visibility, modifier, deprecation-status change
Method & constructor	- Added, removed, renamed, moved, extracted, or inlined - Parameter added, removed; parameter type changed; parameter order changed - No-longer or new outgoing field read/write, method call, class instantiation; no-longer or new incoming call - No-longer or new declared, thrown, and caught exception - Return type change - Visibility, modifier, deprecation-status change
Field	- Added, removed, renamed, or moved - No-longer or new read-by and written-by dependencies - Data type change - Visibility, modifier, deprecation-status change

3.3 Refactoring as Change patterns Queries¹

Once the elementary structural changes between the two versions have been recognized, a suite of queries can be defined and applied to automatically detect a

¹ Due to the space limit, we are not able to give the definition of those queries in this paper. But they have been implemented and are available to download at [36].

wide range of more complex structural change patterns, as compositions of elementary changes, such as composite refactorings [10], or any general structural change patterns of the user's interest.

Table 2. The simple “standard” refactorings

Category	Refactorings
Dealing with containment	• Convert inner type to top-level • Convert top-level type to inner • Move subsystem/package/class
Dealing with generalization	• Pull-up method/field • Push-down method/field • Pull-up behavior • Push-down behavior • Pull-up constructor body
Moving features between objects	• Move method/field • Move behavior
Refactoring class internals	• Rename subsystem/package • Rename class/method/field • Add parameter • Remove parameter • Information hiding • Generalize type • Downcast type • Extract method • Inline method*

The refactorings in Fowler's catalog, henceforth also referred to as “standard” refactorings, are well known and understood and, as such, frequently used in practice. JDevAn is equipped with a set of queries for recognizing instances of these standard refactorings that have occurred in the evolution history of a software system as combinations of design-level change facts reported by UMLDiff. In addition, JDevAn users can define their own queries to its database for any structural change patterns of interest.

We organize the refactorings that can be automatically detected by JDevAn in terms of two separate taxonomies. The first taxonomy classifies refactorings according to their intention and scope: those dealing with changes to the containment hierarchy, those dealing with changes to the inheritance hierarchy, those moving features between objects, and those refactoring class internals.

The second taxonomy examines the complexity of the change and the number and types of change facts that need to be examined in order to recognize the refactoring. Some refactorings are detected directly by

UMLDiff (reviewed in Table 1) since they refer to moves or renamings of design-level entities and their attributes. The refactorings classified as *simple* (reviewed in Table 2) are recognized through queries that examine the UMLDiff change facts. Finally, *composite* refactorings (reviewed in Table 3) are recognized as compositions of elementary structural changes reported by UMLDiff and other simple refactorings.

Table 3. The composite “standard” refactorings

Category	Refactorings
Dealing with containment	• Extract subsystem/package • Inline subsystem/package
Dealing with generalization	• Extract interface • Extract superclass • Extract subclass • Inline superclass • Inline subclass • Form template method • Replace inheritance with delegation • Replace delegation with inheritance
Moving features between objects	• Extract class • Inline class • Die-hard and legacy classes • Convert anonymous class to inner
Refactoring class internals	• Introduce factory method • Introduce parameter object • Encapsulate field • Preserve whole object

In JDevAn, all refactorings and change patterns are detected as compositions of the basic change facts extracted by UMLDiff; no additional “knowledge” is injected in the system at any point after the completion of the UMLDiff process. In principle then, a user has the same information throughout the process. Although, however, the information does not change, we believe that the user's understanding of the rationale of system's design evolution increases. By combining related elementary structural changes into composite refactorings, it becomes easier to understand the potential intent of the change.

4 Case studies

To evaluate the effectiveness of our approach for detecting structural change patterns, we conducted two case studies. Using JDevAn, we analyzed (a) HTMLUnit [34] and (b) JFreeChart [33]. HTMLUnit is a small-size open-source software system for unit testing. There are 11 releases in its history from May 22, 2002 to August 23, 2005. JFreeChart is a medium-size

Java library for drawing charts. It has been under development for more than 4 years and there are 31 major releases between the first version 0.5.6, released on December 1 2000, and the last version 1.0.0, released on November 29 2004. Some of its versions reached more than 800 classes.

Table 4. The number of program entities

	HTMLUnit	JFreeChart
Package	95	698
Class	2639	12866
Interface	128	1686
Field	3239	40829
Method	23779	101311
Constructor	2718	17908
Parameter	11739	142635
Total	44337	317933

Table 5. The number of entity relations

	HTMLUnit	JFreeChart
Contain	36717	205298
Extend	2639	13695
Implement	497	9458
Read	10504	154465
Write	3963	61036
Call	74244	416073
Class usage	17666	165415
Class instantiation	11486	94704
Total	157716	1120144

Table 4 and Table 5 report the numbers of ground entity and relation facts extracted by JDEvAn for the subject systems. Note that only those relevant to the detection of structural change patterns are shown in the tables.

We applied *UMLDiff* to pair-wise compare subsequent releases of the subject systems. The reported structural-evolution changes [27] were stored in the JDEvAn database. Table 6 reports the numbers of discovered change facts. We then manually inspected the correctness of each reported structural change against the system source code, the accompanying Javadocs and source-code comments, and the change logs shipped with each major release version of the system. Erroneously reported and missed changes were corrected through an interactive session with the JDEvAn tool.

Table 7 (presented at the end of the paper) summarizes the number of identified refactorings of each type in the subject systems. For *Extract* and *Inline* refactor-

ings, we count the distinct pairs of the containing or declaring entities of the moved entities and the extracted or inlined methods. For HTMLUnit, the surprisingly large number of *Pull-up* and *Extract superclass* refactorings is the result of refactoring the hierarchy of *HtmlElement* and its 85 descendants. In version 1.3 of HTMLUnit, one top-level superclass *DomNode* and several different intermediate-level superclasses, *StyledElement*, *FocusableElement*, *ClickableElement* were introduced, and many common features, including over 900 *get???Attribute()* methods, that were scattered in all the subclasses in earlier versions were moved to the corresponding superclasses. This massive refactoring resulted in a large amount of moves being identified.

Table 6. The number of Change Facts

Type of change	HTMLUnit	JFreeChart
Entity renaming ²	464	2154
Entity move ³	1098	964
Extract method	254	526
Inline method	27	94
Visibility change	105	845
Data type change ⁴	43	1053
Interface implement change	135	970
Class inheritance change	79	180
Total	2205	6786

Our experience with these case studies has convinced us that JDEvAn's refactoring-detection approach is quite robust to the problem of "multiple different changes occurring to the same part of the system". Many changes at different levels of granularity may have been applied to the same part of the system around the same time, but JDEvAn can still recover them.

Furthermore, we qualitatively evaluated the recall of our refactoring-detection approach against the available change documentations of the subject systems. The results indicate that all of the documented refactorings were indeed recovered [30]. In addition, many undocumented refactorings were also discovered.

Also of interest is the fact that JDEvAn's refactoring reports (in terms of the types of the refactorings and their participants) are more informative than the

² The renamings of methods/constructors include changes of their identifiers and/or modifications to their parameter list (the addition and removal of parameters, the order change of parameters, and the type change of parameters)

³ The moved methods may also have identifier and/or parameter list changes.

⁴ The data type changes include changing field data type, method return type, and parameter type.

plain text in change documentation. This is especially important since much analysis is based on system documentation. For example, Dig and Johnson [5] reported an empirical study on the API evolution of several systems, including Eclipse. Their analysis relies on the changes documented in the release notes shipped with the software system. Compared with the list of refactorings they recovered from the documentation, our refactoring detection approach reports more types and more instances of refactorings; therefore, it might have provided the basis for potentially more informed conclusions. Therefore, we are confident that, although our approach may have missed some of the refactorings, it is able to identify most of the important design changes.

We also computed the percentage of the entities that are affected by the refactoring activities, i.e., the sum of refactoring participants divided by the number of all the entities that ever exist in the evolution of the system, which is shown in the last row of Table 7. We believe this number is interesting for two reasons. First, it shows that refactoring is indeed a common activity in the development process of object-oriented software: about 13-24% entities affected by the refactorings in the evolution of the subject systems. Second, it indicates that our approach can effectively focus the analysis to the relevant parts of the system where the design has been changed: the recovered concrete instances of refactorings and their participants account for only about 13-24% of the whole system.

5 Threats to JDEvAn's refactoring-detection validity

There are several factors that can impact the quality of JDEvAn's structural-change pattern-detection process. It is important to note that in the cases discussed below JDEvAn is usually able to recognize the system elements whose logical structure has evolved, in terms of some general types of refactorings. What is actually impacted is the quality of JDEvAn's inferences regarding the motivation behind the changes. Essentially, it is usually able to recognize the "core" refactorings, but their intentions become difficult to recover. Let us now discuss in some detail these factors individually.

Level of fact-extraction detail: JDEvAn extracts statement-level information regarding the usage dependencies among design entities, such as field read/write, method invocation, object instantiation, etc., which may be resolved through local variables. However, it ignores program-control statements and the usage of local variables, which may affect the identification of some kinds of low-level refactorings, such as *Inline temporary variable*, *Introduce explaining variable*, or *Split temporary variable*, and so on.

These refactorings aim mainly at improving locally the readability and understandability of the code within

a method and they do not affect the overall system's class design as changes that move behavior closer to data, or extract a superclass, or change implementation reuse to interface reuse, etc.

UMLDiff results quality: The detection of structural change patterns relies on the elementary change facts reported by *UMLDiff* when it compares two subsequent system versions. Given a set of correct change facts, the queries we have designed for detecting refactorings will recognize a structural change that conforms to the change pattern advocated by the corresponding refactoring⁵. However, renamings and moves that have been erroneously identified or missed by *UMLDiff* will negatively affect the identification of structural change patterns.

For example, if a method *M* is erroneously identified as renamed, then the refactorings *Introduce parameter object* and *Preserve whole object* involving the method *M* do not make sense any more. On the other hand, if *UMLDiff* misses the moves of methods, fields, and behavior between two classes, the refactoring-detection process may consequently miss the refactoring *Extract class* or some of its partial participants.

Our evaluation of *UMLDiff* has shown its precision and recall to be good in practice [27]. Thus, its negative effects on the detection of structural change patterns should be minor. In principle however, an interactive inspection step could be injected in the process, after the completion of *UMLDiff* and before starting the detection of structural change patterns, to correct the erroneously identified renamings and moves and to identify potentially missed instances.

Consistency of CVS usage: As is the case with the *UMLDiff* accuracy, the quality of the detected change patterns is also affected by the consistence of the CVS usage practices, i.e., how frequently developers save their work in CVS and how much change they tend to bring to the system between two subsequent saves.

For example, before version 1.3 of HTMLUnit, two classes *HtmlFrame* and *HtmlInlineFrame* had almost the same internal structure and they both implemented interface *WebWindow*. In version 1.3 of HTMLUnit, three refactorings were applied to these classes: *Extract superclass BaseFrame*, *Extract class FrameWindow* (the member class of *BaseFrame*) that implements the interface *WebWindow*, and *Replace inheritance with delegation* for *BaseFrame*. Since no intermediate versions were saved, JDEvAn reports the *Extract superclass* and *Extract class* refactorings, but misses the refactoring *Replace inheritance with delegation* since it changed the newly introduced superclass *BaseFrame*

⁵ We cannot theoretically prove correctness, but it has been our experience to date that the refactorings identified by these queries (except for several rare cases) is correct given our own understanding of change patterns.

and its two subclasses. The two subclasses no longer implemented the WebWindow; instead, their superclass BaseFrame newly declared a field of type WebWindow. If an intermediate version were saved after the *Extract superclass* and *Pull-up* the common features refactorings, and then the FrameWindow class was *Extracted* to hold the WebWindow implementation and the *Replace inheritance with delegation* were applied to BaseFrame, all the refactorings would have been recognizable by the JDEvAn queries.

6 Conclusions

In this paper, we presented our method for detecting refactorings in the history of an object-oriented software system, based on design-level analysis. The method, implemented in our JDEvAn tool, relies on three steps. First, facts regarding design-level entities and their relations in each individual version of the system are extracted from the system's source-code versions. Next, the designs of subsequent system versions are pair-wise compared to determine how the basic entities and relations have changed from one version to the next. Finally, queries, corresponding to typical refactorings' change patterns, are applied to the change-facts database to extract instances of particular refactorings and their participant elements.

Our method is general: it is able to recognize a broad range of standard refactorings and it can be extended with special-purpose queries for detecting any change pattern of interest to the user. Our case studies demonstrate that the method is also accurate and precise: it correctly recognizes all refactorings documented by the system developers and more that are not documented, while at the same time correctly focusing on the precise parts of the system that have been affected by the refactoring.

Acknowledgements

This was funded by NSERC.

References

1. I. Balaban, F. Tip and R. Fuhrer. Refactoring support for class library migration. *Proceedings of the 20th OOPSLA*, pp. 265-279, 2005.
2. K. Beck. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 1999.
3. I.D. Baxter and C.W. Pidgeon. Software change through design maintenance. *Proceedings of International Conference on Software Maintenance*, 1997, pp.250-259.
4. J.E. Burge and D.C. Brown. Design rationale for software maintenance. *Proceedings of the 16th International Conference on Automated Software Engineering*, 2001, pp. 433.
5. D. Dig and R. Johnson. The role of refactoring in API evolution. *Proceedings of ICSM*, 2005.
6. S. Demeyer, S. Ducasse and O. Nierstrasz. Finding refactorings via change metrics. *ACM SIGPLAN notices*, 35, 10 (2000), 166-177.
7. A. Dutoit and B. Paech. Rationale management in software engineering. *Handbook on Software Engineering and Knowledge Engineering*, World Scientific, December 2001.
8. S.G. Eick, T.L. Graves, A.F. Karr, J.S. Marron and A. Mockus. Does code decay? Assessing the evidence from change management data. *IEEE Transactions on Software Engineering*, 2001, 27(1):1-12.
9. M. Fischer, M. Pinzger and H. Gall. Populating a release history database from version control and bug tracking systems. *Proceedings of the 19th International Conference on Software Maintenance*, pp. 23-32, September 2003.
10. M. Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
11. E. Gamma, R. Helm, R. Johnson and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
12. M. Godfrey and L. Zou. Using origin analysis to detect merging and splitting of source code entities. *IEEE Transactions on Software Engineering*, 31, 2 (2005), 166-181.
13. C. Gorg and P. Weigerber. Detecting and visualizing refactorings from software archives. *Proceedings of the 13th International Workshop on Program Comprehension*, 2005, pp.205-214.
14. A. Jarczyk, P. Loeffler and I.F. Shipman. Design Rationale for Software Engineering: A Survey. *Proceedings of the 25th Annual IEEE Computer Society Hawaii Conference on System Sciences*, pp. 577-586, January 1992.
15. J. Henkel and A. Diwan. CatchUp! Capturing and replaying refactorings to support API evolution. *Proceedings of the 27th ICSE*, pp. 274-283, 2005.
16. J. Kerievsky. *Refactoring to Patterns*. Addison-Wesley, 2004.
17. M. Lanza. The evolution matrix: Recovering software evolution using software visualization techniques. *Proceedings of the 4th International Workshop on Principles of Software Evolution*, pp. 37-42, 2001.
18. *OMG Unified Modeling Language Specification*, formal/03-03-01, Version 1.5, (2003), <http://www.omg.org>.
19. W.F. Opdyke. *Refactoring Object-Oriented Frameworks*. Ph.D. Thesis, University of Illinois at Urbana-Champaign, 1992.
20. F. Pea-Mora and S. Vadhavkar. Augmenting design patterns with design rationale. *Artificial Intelligence for Engineering Design, Analysis and*

- Manufacturing*, 11, Cambridge University Press, pp. 93-108, 1996.
21. D. Roberts, J. Brant and R.E. Johnson. A refactoring tool for Smalltalk. *Theory and Practice of Object System* 3, 4 (1997), 253–263.
 22. F.V. Rysselberghe and S. Demeyer. Reconstruction of successful software evolution using clone detection. *Proceedings of International Workshop on Principles of software Evolution*, pp. 126–130, September 2003.
 23. C. Schofield, B. Tansey, Z. Xing and E. Stroulia. Digging the development dust for refactorings. *Proceedings of the 14th International Conference on Program Comprehension*, 2006 (to appear).
 24. K. Simon. An improved algorithm for transitive closure on acyclic digraphs. *Theoretical Computer Science* 58, *Automata, Languages and Programming*, 376-386, 1986.
 25. I. Sommerville. Software documentation. In *Software Engineering, vol 2: The supporting Processes*. R.H. Thayer and M.I. Christensen (eds), Willey-IEEE Press.
 26. Z. Xing and E. Stroulia. Recognizing Refactoring from Change Tree, *Proceedings of REFACE (REFactoring: Achievements, Challenges, Effects) workshop in conjunction with the 10th Working Conference on Reverse Engineering*, Nov 13, 2003.
 27. Z. Xing and E. Stroulia. UMLDiff: An algorithm for object-oriented design differencing. *Proceedings of the 20th International Conference on Automated Software Engineering*, 2005.
 28. Z. Xing and E. Stroulia. Understanding class evolution in object-oriented software. *Proceedings of the 12th International Workshop on Program Comprehension*, pp. 34-43, June 2004.
 29. Z. Xing and E. Stroulia. Towards Mentoring Object-Oriented Evolutionary Development. *Proceedings of the 21st International Conference on Software Maintenance*, 2005.
 30. C. Schofield, B. Tansey, Z. Xing and E. Stroulia. Digging the development dust for refactorings. *Proceedings of the 14th International Conference on Program Comprehension*, 2006.
 31. Z. Xing and E. Stroulia. Refactoring practice: How it is and how it should be supported – An Eclipse case study. *Proceedings of the 22nd International Conference on Software Maintenance*, 2006 (to appear).
 32. Eclipse, www.eclipse.org
 33. JFreeChart: www.jfree.org/jfreechart/
 34. HTMLUnit: htmlunit.sourceforge.net/
 35. IntelliJ IDEA, www.jetbrains.com/idea/features/refactoring.html
 36. JDEvAn, <http://www.cs.ualberta.ca/~xing/jdevan.html>

Table 7. The summary of identified refactorings

Type of refactoring	HTMLUnit	JFreeChart
Convert inner type to top-level	1	1
Convert top-level to inner	0	1
Extract subsystem	0	0 ⁶
Inline subsystem	0	0
Extract package	1	15
Inline package	4	1
Pull-up method/field	1008	315
Push-down method/field	22	52 ⁷
Pull-up behavior	141	165
Push-down behavior	6	7
Pull-up constructor body	3	9
Extract interface	8	55
Extract superclass	119	11
Extract subclass	1	3
Inline superclass	0	2
Inline subclass	2	23 ⁸
Form template method	1	3
Replace inheritance with delegation	1	5
Replace delegation with inheritance	0	0
Extract class	13	34
Inline class	2	0
Die-hard/legacy classes	0	8
Convert anonymous class to nested	1	0
Move method/field	32	216
Move behavior	71	102
Deprecation + delegation	14	83
Information hiding	7	707
Generalize type	6	140
Downcast type	10	54
Introduce factory method	0	0
Introduce parameter object	0	0
Encapsulate field	9	182
Preserve whole object	0	4
Total	1483	2197
Focus %	≈24%	≈13%

⁶ HTMLUnit and JFreeChart have only one top-level subsystem. Thus, they have no *Extract/Inline subsystem* refactorings.

⁷ Includes 35 static final constants being inlined from the interfaces into the classes that implement them in the earlier versions.

⁸ 16 out of 23 were the results of the redesign of the axis, plot, and render hierarchies in order to eliminate the parallel inheritance smells.